System Level Security Measures Implemented in OpenBSD[1]

B. Tech Seminar Report

Anirudha Bose [2] [B112006]
Department of Computer Science & Engineering
International Institute of Information Technology, Bhubaneswar
IIIT Bhubaneswar

Last updated on: May 28, 2015
Cite as: IIITBH-ACM/TR/2015/003
Permalink: http://iiitbh-acm.github.io/IIITBH-ACM_TR_2015_003.pdf

[1]Supplementary technical report in support of the seminar presented in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in Computer Science & Engineering.

[2]Electronic Address: ani07nov@gmail.com

Undertaking

I declare that the work presented in this thesis titled "System Level Security Measures Implemented in OpenBSD", submitted to the Department of Computer Science & Engineering, International Institute of Information Technology, Bhubaneswar, for the award of the Bachelors of Technology degree in Computer Science & Engineering, is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

Student Signature:
Anirudha Bose
Department of Computer Science & Engineering
IIIT Bhubaneswar

Certificate

Certified that the work contained in the thesis titled "System Level Security Measures Implemented in OpenBSD", by Anirudha Bose, Roll Number B112006 has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Faculty Signature:
Prof. Rakesh Kumar Lenka
Department of Computer Science & Engineering
IIIT Bhubaneswar

Abstract

This paper is a survey of the security measures implemented in OpenBSD at a system level. OpenBSD is claimed to be the most secure operating system in the world due to its defensive technologies which makes it extremely hard to write an exploit. Recent developments in OpenBSD make the system environment much more hostile towards exploitation, without impacting well behaving processes. The techniques outlined range from safe string handling, to innovative Address Space Layout Randomization techniques, Position Independent Executables, OpenBSD StackGap, Stack Smashing Protection by ProPolice, and randomization in runtime linker, mmap(2) and malloc(3). Several compiler improvements originally implemented by OpenBSD have also been presented. The paper finally concludes by pointing out some of the limitations in user level protection that need to be addressed in future.

Acknowledgements

I would like to thank Prof. Rakesh Kumar Lenka, Assistant Professor, Department of Computer Science & Engineering, IIIT Bhubaneswar for providing useful inputs to improve the content and format of the report and reviewing the final work.

Contents

Abstract **iii**

Acknowledgements **iv**

1 Introduction **1**

2 Goals and Objectives of OpenBSD **2**

3 System Level security measures **3**

3.1 Safe string handling with strlcpy() and strlcat() 3
3.2 Position Independent Executables 3
3.3 $W \oplus X$ Policy 4
3.4 OpenBSD StackGap 4
3.5 ProPolice/SSP 4
3.6 Static checks of bounds 5
3.7 The .rodata segment 5
3.8 Random numbers in OpenBSD 5
3.8.1 Randomness in the run-time linker 5
3.8.2 Randomness in mmap(2) 6
3.8.3 Randomness in malloc(3) 6

4 Conclusion **7**

1 Introduction

OpenBSD is a Unix-like operating system derived from Berkeley Software Distribution (BSD) 4.4, a UNIX derivative developed at the University of California, Berkeley. It was originally created as a fork of NetBSD by project leader Theo de Raadt in late 1995.

OpenBSD is ultra-secure and elegant in design. It is a result of heavy research, and based on some of the best UNIX operating systems of the past. All features of modern operating systems can be found in OpenBSD. OpenBSD is widely considered as one of the most secure general-purpose operating systems available today and it supports many key parts of the global Internet infrastructure.

2 Goals and Objectives of OpenBSD

OpenBSD places emphasis on code correctness, reliability, security, standardization, and portability. It modifies UNIX to be more resilient to attacks. Therefore a great deal of attention is paid to security problems in order to fix them before anyone else does. Subsystems in OpenBSD are always designed for simplicity. Peer review of patches is mandatory at every level: design, coding, and even for simple changes.

The source tree integrates good code from any source with acceptable (BSD) licenses. The Foundation strives to make software robust and secure, and encourages companies to use whichever pieces they want to. There are many commercial spin-offs of OpenBSD.

Standards like ANSI and POSIX are tracked and implemented for greater portability and cross-platform reliability. One of the major goals is to work towards a very machine independent source tree and support as many different systems and hardware as feasible. Popular hardware architectures supported are i386, alpha, amd64, arm, macppc, sparc, sparc64, sgi, vax, socppc, octeon, hppa, landisk.

OpenBSD provides a good set of security tools for SSH, SSL, IPSEC, and like. Most user-land utilities are patched separately by package maintainers. Non-required services are not activated in a default installation. Default configuration of services are adjusted with security considerations in mind. Activating a service requires a manual action of the system administrator.
OpenBSD Foundation makes a new release of OpenBSD approximately every six months. The aim is to provide a working, functional, and useful system. The result is that it is the #1 most secure operating system, with only 2 remote vulnerabilities discovered in more than 10 years.

3 System Level security measures

3.1 Safe string handling with strlcpy() and strlcat()

The `strlcpy()` and `strlcat()` functions first appeared in OpenBSD 2.4 as a replacement for length-bounded string functions such as `strncpy()` and `strncat()`. Use of standard C string functions `strcpy()` and `strcat()` is not recommended due to prevalence of buffer overflow attacks. The new API in OpenBSD serves as an alternate, and intuitive method with safe string copies in mind [2].

The proposed replacement functions, `strlcpy()` and `strlcat()` guarantee that the strings are `NUL` terminated and take as a length parameter the size of the string in bytes. The API also provides an easy way to detect truncation. Unlike `strncpy()` and `strncat()`, neither function zero-fills the unused bytes in the destination string.

Example 1: Code fragment using strcpy() and strcat()

```
strcpy(path, homedir);
strcat(path, "/");
strcat(path, ".foorc");
len = strlen(path);
```

Example 2: Converted to strncpy() and strncat()

```
strncpy(path, homedir, sizeof(path) - 1);
path[sizeof(path) - 1] = '\ 0';
strncat(path, "/", sizeof(path) - strlen(path) - 1);
strncat(path, ".foorc", sizeof(path) - strlen(path) - 1);
len = strlen(path);
```

Example 3: Trivial conversion to strlcpy()/strlcat()

```
strlcpy(path, homedir, sizeof(path));
strlcat(path, "/", sizeof(path));
strlcat(path, ".foorc", sizeof(path));
len = strlen(path);
```

3.2 Position Independent Executables

Position Independent Executables (PIE) are an output of the hardened package build process. A PIE binary and all of its dependencies are loaded into random locations within virtual memory each time the application is executed. This makes Return Oriented Programming (ROP) attacks much more difficult to execute reliably.

With support for Position Independent Executable (PIE), the virtual memory address space has an improved address space layout randomization, allowing the main program .text segment to have a variable address[3]. The other shared libraries still have random base addresses as in ordinary ASLR techniques. This feature in OpenBSD prevents most buffer overflow attacks like the return-to-libc attack on a non-executable stack. Position Independent Executable (PIE) implementation in OpenBSD however results in larger and slower binaries, is buggy, and not supported well on all platforms.

3.3 $W \oplus X$ Policy

Many exploits rely on the fact that address space has memory which is both writable as well as executable (W and X). There is a need to have a generic policy for the whole address space which also conforms to the standards like POSIX and ANSI.

The $W \oplus X$ policy states that a page may be both writable or executable but not both, hence the name Write XOR Execute[1, 3, 4, 6]. Such a feature is easy to implement on architectures with fine grained page permissions, i.e. architectures supporting per-page X bit like X86_64, SPARC, SPARC64, ARM, etc. On other architectures like i386 and PowerPC, it is not so straightforward to implement $W \oplus X$. For some models like VAX, MIPS, and M68k, it is impossible. Thus, this policy cannot be implemented on all platforms.

On an Intel i386 machine, there is no NX bit in page tables. The range of execution is demarcated by an `exec` line at 512 MB of the virtual address space. There is also a gap of 512MB between the base address of `.text` and base address of `.data`. The user programs are loaded high at a fixed address and heap follows the `.data` segment (emulating the NX bit feature) and a gap is left below the `exec` line for the executable pages. Thus, when the `.text` segment of the `ld.so` and other shared libraries are loaded into random addresses below exec line, the `.data` is guaranteed to be in non-executable region.

3.4 OpenBSD StackGap

Stack overflows are the easiest exploitable vulnerability. A stack overflow occurs if the stack pointer exceeds the stack bound since the call stack consists of a limited amount of address space. In a stack based buffer overflow, attacker overflows the buffer on stack. Overflow overwrites function return address, and hence the control jumps to the buffer where the attack code (shellcode) is placed. The solution is to keep a random sized gap at the top of stack. This mechanism is called the OpenBSD StackGap. This makes it very difficult to predict the location of the function return address to be overwritten by the malicious shellcode. StackGap uses at most 1 page of real memory, so the space overhead occupied in stack is very low. Range of the random gap in the stack can be controlled with the `kern.stackgap_random` sysctl. A value of 0 disables it. Otherwise, it has to be a power of 2 and not too large. The default value is 64K.

3.5 ProPolice/SSP

Stack Smashing Protector (SSP) is a security extension for the GCC. Developed by IBM, the SSP protects against the single largest class of stack smashing exploits. GCC modified the original SSP implementation to add SSP to Thread Local Storage (TLS), so that each thread can be guarded independently. The GCC options for SSP are `-fstack-protector`, `-fstack-protector-all`, and `-Wstack-protector`. The `--param=ssp-buffer-size=` GCC option controls the minimum buffer size protected by SSP.

ProPolice is a modification to the GCC in which a random canary is stored on the stack by the function prologue. The protection code is injected into each candidate function at compile time. The function epilogue then validates if the canary has been modified by an overflowed buffer before the function returns. In the event of a dirtied canary, the compiler must call the `abort()`. ProPolice was integrated into OpenBSD in December 2002, and now a part of GCC since version 4.1. GCC with Propolice (SSP) is activated by default

to build all libraries and applications in OpenBSD.

3.6 Static checks of bounds

Static checking of bounds for overflows was originally implemented by OpenBSD, and now merged upstream (GCC, all other modern compilers). `-Wbounded` is a compiler option that performs static checking to make sure the bounds length passed to common functions matches the real buffer length. Functions must be marked with the new attribute `__bounded__`, and warnings are turned on by `-Wbounded`.

```
char buf[1024]; /* buf size fixed at compile time */
snprintf(buf, SIZE, fmt); /* -Wbounded will check if SIZE == 1024 */
```

3.7 The .rodata segment

Read-only strings and pointers which were originally stored in the `.text` segment had the attributes `PROT_READ` and `PROT_EXEC`, allowing constant data to be executed by a potential attacker. The idea is to create a new `.rodata` segment with `PROT_READ` attribute only thus losing the `PROT_EXEC` attribute. This mechanism ensures that objects in the address space have the minimal set of permissions.

3.8 Random numbers in OpenBSD

Good random numbers are fundamental to all secure computer systems. Cryptographically secure pseudo-random number generators use a random seed value as a starting point. On most Unix-like systems, including Linux, FreeBSD, and NetBSD, the pseudo device file `/dev/random` will block until sufficient entropy is harvested from the environment. OpenBSD puts special focus on using a strong source of randomness in order to reduce the predictibiity of the system. OpenBSD has a centralized random number generation system which improves security since it is becomes very hard to observe or predict.

Entropy can be gather from various I/O devices such as keyboard, mouse, NIC, and audio cards. Entropy can also be gathered from hardware sources like the CPU itself, or some dedicated crypto chips which have very high efficiency. OpenBSD implements a function `arc4random()`, its own home grown random number generator to provide higher quality data than those described in `rand(3)`, `random(3)`, and `rand48(3)`. Use of these functions is encouraged for almost all random number consumption, and third-party utilities. Other interfaces are deficient in either quality, portability, standardization, or availability.

3.8.1 Randomness in the run-time linker

The program `ld.so` finds and loads the shared libraries needed by a program, prepares the program to run, and then runs it. OpenBSD's version of `ld.so` loads libraries randomly in memory to randomize the shared library order. To achieve this, OpenBSD makes use of `arc4random()` to generate the required random addresses. This is a much needed feature for large programs which load at lot of libraries at runtime, like sshd, and Mozilla services to name a few.

The randomized `ld.so` attempts to map shared libraries randomly into the address space by skewing their base address. The shared libraries are also mapped in a different order on every load to the memory. This makes return to libc attacks a lot more difficult since it is no more possible to determine the address of libc functions like `system()` and `exit()`.

This improved security however comes at the cost of some amount of virtual address space which is unused.

3.8.2 Randomness in mmap(2)

`mmap(2)` is a system call to map files or devices into memory. `mmap(2)` creates a new mapping of a file descriptor in the virtual address space of the calling process. The starting address for the new mapping is specified in `addr`. The `length` argument specifies the length of the mapping.

According to POSIX specifications, if the `MAP_FIXED` flag is specified, the allocation will happen at the specified address, replacing any previously established mappings in its range. If however, `MAP_FIXED` is not specified, the operating system is free to select any memory location. Other operating systems do this very predictably, but OpenBSD adds a security mechanism by randomizing this address. Each time the program is run, there is a different address space behavior and thus it is not possible to guess the location of a function or code placed in the virtual memory.

3.8.3 Randomness in malloc(3)

Addresses of objects allocated by `malloc(3)` are fairly predictable, and many of the recent heap attacks have relied on this vulnerability. The new version of `malloc(3)` adopted by OpenBSD ensures randomization of this allocated address. If the allocation is smaller than `PAGE_SIZE`, `malloc()`, which maintains buckets of chunks, randomizes chunk selection from the buckets. However, if the allocation is greater than or equal to the `PAGE_SIZE`, then `malloc(3)` relies on the randomness of `mmap(2)`.

Furthermore, OpenBSD implements the Guard Page mechanism in which allocations in heap memory are followed by a guard page. If there is any attempt to overflow the allocation and read the guard page, the compiler throws a segmentation fault. Guard Pages can be activated by setting the `G` option in `/etc/malloc.conf`. As a result of these two features, heap attacks are very difficult to pull off on an OpenBSD box.

4 Conclusion

This technical paper discussed some of the system level security mechanisms implemented in OpenBSD to provide a safe and robust operating system. The paper outlined several compiler features, memory protection techniques, use of randomization, and page guarding techniques. Network level protection measures were not within the scope of this paper, though OpenBSD provides unique security features in its network stack.

What is missing in OpenBSD is user application protection. Userland softwares like web browsers, multimedia viewers and players, email clients are responsible for their own security implementations. Some sandboxing techniques could well be implemented in OpenBSD in future. Another problem that needs to be addresses is the pool of kernel-level privileged code of X Server running in the user-space. Privilege separation is a solution, but is unfortunately not enough.

OpenBSD is a hostile environment for user applications. The open source community of OpenBSD is working actively to make it the most secure operating system. It is imperative that other operating systems adopt similar technologies such that third party applications comply with the security standards followed by OpenBSD.

References

[1] Peter Valchev, *Using OpenBSD Security Features to Find Software Bugs*, ACM Reflections/Projections (2007).

[2] Todd C. Miller, Theo de Raadt, *strlcpy and strlcat consistent, safe, string copy and concatenation*, USENIX (1999).

[3] Kurt Miller, *Implementing Position Independent Executables (PIE) on OpenBSD*, DCBSDCon (2009).

[4] Theo de Raadt, *Exploit Mitigation Techniques*, OpenCON (2005)

[5] Otto Moerbeek, *A new malloc(3) for OpenBSD*, EuroBSDCon (2009).

[6] Matthieu Herrb, *Protection measures in OpenBSD*, H2K9 (2009).